รายงาน Trigger ในฐานข้อมูลระบบ UP Cloth Market

รายวิชา 227202 Database · กลุ่ม “ไม่ใส่กุ้ง”

ฐานข้อมูล: Supabase (PostgreSQL)

1. ภาพรวม

ระบบ UP Cloth Market ใช้ Supabase เป็นฐานข้อมูล PostgreSQL หน้าเว็บเป็น Vue 3 ที่เรียก REST API ของ Supabase (PostgREST) โดยตรง ไม่มีเซิร์ฟเวอร์ของกลุ่มคั่นกลาง ผลคือผู้ใช้ที่ล็อกอินแล้วมี token อยู่ในมือ และส่งคำขอตรงไปที่ API ได้เองโดยไม่ต้องผ่าน หน้าเว็บ กฎสำคัญของระบบจึงต้องบังคับที่ฐานข้อมูล ไม่ใช่แค่ซ่อนปุ่มบนหน้าเว็บ

ระบบป้องกันที่ฐานข้อมูลมี 2 ชั้นทำงานคู่กัน ชั้นแรกคือ RLS (Row Level Security) ตัดสินว่าผู้ใช้คนนี้อ่านหรือแก้แถวไหนได้ ชั้นที่สองคือ Trigger ตัดสินว่า ค่าในแถวเปลี่ยนแบบไหนได้ และค่าไหนฐานข้อมูลต้องเติมให้เอง ซึ่ง RLS ทำไม่ได้ เพราะ RLS ตรวจทั้งแถว แยกตรวจทีละคอลัมน์ไม่ได้ และไม่ได้เทียบค่าเก่ากับค่าใหม่ (ข้อยกเว้น: Trigger ของระบบนี้ไม่มีตัวไหนใช้ยกเลิกคำสั่ง ทุกตัวเป็นแบบเติมข้อมูล/แจ้งเตือนเท่านั้น รายละเอียดในข้อสังเกตของแต่ละบท)

Trigger คือฟังก์ชันที่ PostgreSQL เรียกให้อัตโนมัติเมื่อเกิด INSERT, UPDATE หรือ DELETE บนตาราง ผูกกับตารางด้วยคำสั่ง CREATE TRIGGER และเรียกฟังก์ชันชนิด RETURNS trigger ที่เขียนด้วยภาษา PL/pgSQL ภายในฟังก์ชันใช้ตัวแปร NEW แทนค่าแถวใหม่ และ OLD แทนค่าแถวเดิม

ฐานข้อมูลของกลุ่มมี Trigger ที่กลุ่มสร้างเอง 5 ตัว ดังนี้

#	Trigger	ตาราง	ทำงานเมื่อ	ฟังก์ชัน	หน้าที่
1	trg_order_item_notify	order_item	AFTER INSERT	notify_user()	แจ้งผู้ขายว่ามีออเดอร์ใหม่
2	trg_orders_status_notify	orders	AFTER UPDATE	notify_user()	แจ้งผู้ซื้อเมื่อสถานะออเดอร์เปลี่ยน (ข้อความเป็นภาษาไทย)
3	trg_orders_insert_notify	orders	AFTER INSERT	notify_user()	แจ้งผู้ซื้อว่า “บันทึกคำสั่งซื้อแล้ว”
4	trg_refresh_product_quantity	product_variant	AFTER INSERT / UPDATE / DELETE	refresh_product_quantity()	ยอดรวม product.quantity = ผลรวมสต็อกทุก variant เสมอ
5	trg_messages_notify	messages	AFTER INSERT	notify_user()	แจ้งอีกฝ่ายเมื่อมีข้อความแชทใหม่ + ลิงก์เข้าห้อง

[FIG 1: รายการ Trigger ทั้งหมดในฐานข้อมูลจริง ดึงจาก pg_trigger ผ่าน Supabase SQL Editor คำสั่งดู: select tgname, tgrelid::regclass, tgenabled from pg_trigger where not tgisinternal order by tgrelid::regclass::text, tgname;]

1.1 ลักษณะร่วมของทั้ง 5 ตัว

- FOR EACH ROW ทำงานทีละแถว ถ้าคำสั่งเดียวเพิ่ม order_item 3 แถว Trigger จะถูกเรียก 3 ครั้ง และอ่าน

NEW/OLD ของแต่ละแถวได้ (ผลข้างเคียงที่ตั้งใจ: ออเดอร์ 1 ใบที่มีสินค้า 3 ร้าน จะเกิดแจ้งเตือน “มีออเดอร์ใหม่” 3 ครั้ง ครั้งละร้าน)

- AFTER ทั้ง 5 ตัว เพราะทุกตัวต้องอ่านค่าหลังบันทึกแล้ว (อ่านแถว NEW ที่เขียนลงตาราง แล้ว หรือ sum ยอดจากแถวเพื่อนบ้านที่ commit แล้ว) ไม่มีตัวไหนต้องแก้ค่าใน NEW ก่อนบันทึกหรือยกเลิกคำสั่ง จึงไม่ต้องใช้ BEFORE
- SECURITY DEFINER ฟังก์ชันรันด้วยสิทธิ์ของเจ้าของฟังก์ชัน ไม่ใช่สิทธิ์ของผู้ที่ส่งคำสั่ง จึงเขียนตาราง notifications และอ่านตาราง product/product_variant ได้ครบ แม้ RLS ของผู้ใช้คนนั้นจะมองไม่เห็น (เช่น ตาราง notifications ไม่มี policy สำหรับ INSERT เลย การเขียนแจ้งเตือนจึงทำได้เฉพาะทาง Trigger/RPC เท่านั้น นี่คือการออกแบบ ไม่ใช่ช่องโหว่)
- SET search_path TO 'public' ล็อกให้ชื่อตารางในฟังก์ชันหมายถึง schema public เสมอ กันการสร้างตารางหรือฟังก์ชันชื่อซ้ำใน schema อื่นมาหลอกฟังก์ชันที่รันด้วยสิทธิ์สูง ซึ่งเป็นข้อแนะนำด้านความปลอดภัยของ Supabase สำหรับฟังก์ชัน SECURITY DEFINER
- ไม่มี RAISE EXCEPTION เลย ต่างจาก Trigger สาย guard ทั่วไป ระบบนี้เลือกให้ Trigger ล้มเหลวไม่ได้ (ถ้า notify_user() พัง ออเดอร์ต้องยังสำเร็จ) ส่วนการตรวจสต็อก และการตัดสต็อกอยู่ใน RPC place_order() ซึ่งรันเป็น transaction เดียวกัน: ถ้าตรวจไม่ผ่านทั้งออเดอร์รวมทั้ง notification ที่สร้างไปแล้วจะ rollback หมด

ฟังก์ชันช่วยที่ Trigger เรียกใช้

ฟังก์ชัน	คืนค่า	ความหมาย
auth.uid()	uuid	ฟังก์ชันของ Supabase คืน id ของผู้ใช้จาก JWT ที่แนบมากับคำขอ ถ้าไม่มีผู้ใช้ล็อกอิน (เช่น คำสั่งจาก SQL Editor) จะได้ NULL
notify_user()	trigger	ฟังก์ชันกลางของ Trigger ตัวที่ 1–3 แยกสาขาตาม TG_TABLE_NAME และ TG_OP
refresh_product_quantity()	trigger	คำนวณ SUM(quantity) ของ variants แล้วเขียนลง product.quantity
place_order(uuid, jsonb, text)	int	RPC สร้างออเดอร์แบบ atomic (ตรวจ+ตัดสต็อกทีละแถว) คืน order_id
is_own_order(int)	boolean	เช็กว่าออเดอร์เป็นของผู้เรียกหรือไม่ แบบข้าม RLS (SECURITY DEFINER) ใช้ใน policy เพื่อตัดวงจร RLS ระหว่าง orders กับ order_item

2. Trigger 1: trg_order_item_notify

2.1 หน้าที่และเหตุผล

เมื่อมีแถวใหม่ใน order_item (สินค้าชิ้นหนึ่งในออเดอร์) ต้องบอกผู้ขายเจ้าของสินค้านั้น ทันทีว่า “มีออเดอร์ใหม่” เพราะผู้ขายไม่มีทางรู้เองว่ามีคนสั่งของร้านตน เหตุที่ใช้ Trigger แทนการให้หน้าเว็บ insert แจ้งเตือนเองมี 2 ข้อ คือ (1) ตาราง notifications ไม่มี INSERT policy หน้าเว็บเขียนตรงไม่ได้อยู่แล้ว (2) ให้หน้าเว็บ ส่งมาก็เชื่อไม่ได้ ผู้ใช้อาจส่งแจ้งเตือนปลอมหาคนอื่นได้

นิยามใน repo: supabase/migrations/0001_schema.sql:158-161 (ฟังก์ชัน notify_user() สาขา order_item)

```
CREATE TRIGGER trg_order_item_notify
AFTER INSERT ON public.order_item
FOR EACH ROW EXECUTE FUNCTION public.notify_user();
-- ใน notify_user(): สาขา tg_table_name = 'order_item'
-- insert into notifications (user_id, type, title, content)
-- select p.seller_id, 'order', 'มีออเดอร์ใหม่',
--        'มีคำสั่งซื้อ #' || new.order_id || ' สำหรับสินค้า ' || p.product_name
```

```
-- from product p where p.product_id = new.product_id;
```

2.2 อธิบายโค้ด

1. อ่านเจ้าของสินค้าจากแถวใหม่ JOIN ตาราง product ด้วย new.product_id แล้วใช้ seller_id นั้นเป็นผู้รับแจ้งเตือน ไม่เชื่อค่าที่คำขอส่งมา
2. type = 'order' ตรงกับไอคอนในหน้าแจ้งเตือน (NotificationsPage.vue)
3. 1 แถว order_item = 1 แจ้งเตือน ถ้าออเดอร์เดียวมีของ 3 ร้าน จะเกิด 3 แถว ครั้งละร้าน ถูกต้องตามออกแบบ

2.3 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่คาด (จากโค้ด)	สถานะการยืนยัน
1.1 สั่งซื้อ 1 ออเดอร์มีของ 1 ร้าน	สั่งผ่านแอป แล้วดู notifications ของ seller	มีแถว type='order' 1 แถว ชื่อสินค้าถูก	ยืนยันแล้ว (ออเดอร์ #11 ฝั่งผู้ขายเห็นและแจ้งเตือนขึ้น)
1.2 ออเดอร์เดียวมีของ 2 ร้าน	สั่งของ 2 ร้านในออเดอร์เดียว	เกิด 2 แถว ครั้งละ seller_id	รอผลรัน
1.3 product_id ไม่มีจริง	แทรก order_item ชี้สินค้าที่ไม่มี	select ... from product ได้ว่าง ไม่เกิดแจ้งเตือน แต่แถวถูกเขียน (FK จะกันไว้ก่อนอยู่แล้ว)	ตามโค้ด

[FIG 2: กระดิ่งแจ้งเตือนฝั่งผู้ขายขึ้น “มีออเดอร์ใหม่” หลังผู้ซื้อกดสั่ง]

2.4 หน้าจอที่เกี่ยวข้อง

หน้า “คำสั่งซื้อ” ฝั่งผู้ขาย (SellerOrdersPage.vue) + กระดิ่งใน SellerLayout.vue (โหลด + subscribe realtime อัตโนมัติเมื่อเปิดโหมดผู้ขาย)

2.5 ข้อสังเกต

- แจ้งเตือนผูกกับ seller_id ณ เวลาสั่ง ถ้าย้ายสินค้าไปร้านอื่นทีหลัง ประวัติยังชี้ร้านเดิม ถูกต้องแล้วเพราะเป็น snapshot เชิงตรรกะ

3. Trigger 2: trg_orders_status_notify

3.1 หน้าที่และเหตุผล

เมื่อสถานะออเดอร์เปลี่ยน (ผู้ขายกด “อัปเดตเป็น จัดเตรียมสินค้า” ฯลฯ) ต้องบอกผู้ซื้อ
เพราะผู้ซื้อไม่มีทางรู้เองว่าของถึงขั้นไหน เหตุที่ต้องมี Trigger ตรวจว่า “เปลี่ยนจริง” ก่อนแจ้ง เพราะ RPC place_order() มีคำสั่ง UPDATE orders SET total_amount = ... ทุกครั้งหลังสร้างออเดอร์ ถ้าไม่เช็ก OLD เทียบ NEW
ผู้ซื้อจะได้แจ้งเตือนขยะทุกออเดอร์

นิยามใน repo: supabase/migrations/0001_schema.sql:163-166 (สาขา orders + UPDATE ใน notify_user(), ป้ายไทยเติมใน 0008_order_status_pipeline.sql)

```
CREATE TRIGGER trg_orders_status_notify
AFTER UPDATE ON public.orders
FOR EACH ROW EXECUTE FUNCTION public.notify_user();
-- ใน notify_user(): สาขา tg_table_name = 'orders' และ TG_OP = 'UPDATE'
-- elsif old.status is distinct from new.status then
--   แปลง new.status เป็นป้ายไทยด้วย CASE
--   (Pending=รอดำเนินการ, Preparing=จัดเตรียมสินค้า,
--    Shipped=จัดส่งแล้ว, Delivered=ส่งมอบแล้ว)
```

```
--   insert ... values (new.buyer_id, 'order_status',
--     'สถานะคำสั่งซื้อ #' || new.order_id || ' อัปเดต',
--     'สถานะเปลี่ยนเป็น ' || v_label);
```

3.2 อธิบายโค้ด

1. ใช้ IS DISTINCT FROM แทน <> เพื่อให้เทียบค่า NULL ได้ถูกต้อง
2. แปลงรหัสสถานะอังกฤษเป็นป้ายไทยด้วย CASE ตั้งแต่ชั้น DB หน้าเว็บแสดงข้อความได้เลย ไม่ต้อง map ซ้ำ
3. ถ้า UPDATE คอลัมน์อื่น (เช่น total_amount) โดย status เท่าเดิม จะเงียบ ไม่สร้างแจ้งเตือน

3.3 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่คาด (จากโค้ด)	สถานะการยืนยัน
2.1 เปลี่ยน Pending Preparing	ผู้ขายกดอัปเดต แล้วดู notifications ของ buyer	มีแถว type='order_status' ข้อความ “สถานะเปลี่ยนเป็น จัดเตรียมสินค้า”	รอผลรัน
2.2 UPDATE แต่ status เท่าเดิม	update orders set total_amount = total_amount where ...	ไม่มีแถวใหม่	ตามโค้ด
2.3 เปลี่ยนย้อนขั้น (เช่น Shipped Preparing)	หน้าเว็บกันไว้ (ปุ่มมีแค่ขั้นถัดไป) แต่ถ้ายิง API ตรง	Trigger ไม่กันขั้น (กันที่ UI) ยังสร้างแจ้งเตือนตามค่าจริง	ตามโค้ด มีข้อสังเกต

[FIG 3: กระดิ่งผู้ซื้อขึ้นข้อความภาษาไทยหลังผู้ขายอัปเดตสถานะ]

3.4 หน้าจอที่เกี่ยวข้อง

ป้ายสถานะทุกหน้าใช้ StatusBadge.vue + STATUS_LABEL ส่วนปุ่มอัปเดตอยู่หน้า “คำสั่งซื้อ” ฝั่งผู้ขาย (ไม่มี dropdown ข้ามขั้น มีแค่ปุ่มขั้นถัดไป)

3.5 ข้อสังเกต

- กรณี 2.3 Trigger ไม่ตรวจลำดับขั้น การกันย้อนขั้นอยู่ที่ UI อย่างเดียว ถ้าต้องการกันระดับ DB ต้องเพิ่ม BEFORE UPDATE trigger เทียบลำดับขั้น (ยังไม่มีในระบบนี้)

---

4. Trigger 3: trg_orders_insert_notify

4.1 หน้าที่และเหตุผล

เมื่อสร้างออเดอร์สำเร็จ (INSERT ลง orders) ต้องบอกผู้ซื้อว่า “บันทึกคำสั่งซื้อแล้ว” พร้อมสถานะเริ่มต้น ไม่งั้นผู้ซื้อกดสั่งแล้วเงียบ ไม่รู้ว่าสำเร็จไหม (ต้องรอเปิดดูประวัติเอง) Trigger นี้เพิ่มใน 0008 หลังจากพบว่าของเดิมมีแค่แจ้งเตือนผู้ขายกับแจ้งเตือนตอน เปลี่ยนสถานะ แต่ไม่มีอะไรบอกผู้ซื้อตอนกดสั่งพอดี

นิยามใน repo: supabase/migrations/0008_order_status_pipeline.sql (สาขา orders + TG_OP = 'INSERT' ใน notify_user())

```
drop trigger if exists trg_orders_insert_notify on public.orders;
create trigger trg_orders_insert_notify
after insert on public.orders
for each row execute function public.notify_user();
-- ใน notify_user(): สาขา tg_table_name = 'orders' และ TG_OP = 'INSERT'
-- insert ... values (new.buyer_id, 'order_placed',
--   'บันทึกคำสั่งซื้อ #' || new.order_id || ' แล้ว',
```

```
--  'คำสั่งซื้อของคุณถูกบันทึกแล้ว สถานะปัจจุบัน: รอดำเนินการ');
```

4.2 อธิบายโค้ด

1. ใช้ TG_OP = 'INSERT' แยกสาขาชัดเจน ไม่ปนกับสาขา UPDATE (กันแจ้งเตือนซ้ำ)
2. ข้อความสถานะเขียนตายตัวว่า "รอดำเนินการ" เพราะ place_order() สร้างออเดอร์ ด้วย status Pending เสมอ
3. ทำงานใน transaction เดียวกับ place_order() ถ้าสต็อกไม่พอแล้วทั้งออเดอร์ rollback แจ้งเตือนนี้จะหายไปด้วย (ไม่มีแจ้งเตือนค้างของออเดอร์ที่สร้างไม่สำเร็จ)

4.3 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่คาด (จากโค้ด)	สถานะการยืนยัน
3.1 สั่งซื้อสำเร็จ 1 ออเดอร์	สั่งผ่านแอป แล้วดู notifications ของ buyer	มีแถว type='order_placed' 1 แถว	รอผลรัน
3.2 สั่งแล้วสต็อกไม่พอ (RPC error)	สั่งเกินสต็อก	ไม่มีแถวใหม่ (rollback ทั้งก้อน)	ตามโค้ด

[FIG 4: กระดิ่งผู้ซื้อขึ้น "บันทึกคำสั่งซื้อ #N แล้ว" ทันทีหลังกดยืนยัน]

4.4 หน้าจอที่เกี่ยวข้อง

Toast "สั่งซื้อสำเร็จ!" ใน CheckoutPage.vue (แจ้งแบบชั่วคราว) คู่กับ notification แถวนี้ใน DB (แจ้งแบบถาวรดูย้อนหลังได้)

4.5 ข้อสังเกต

- seed (0002_seed.sql) มีออเดอร์ตัวอย่าง 5 ใบ การรัน seed จะสร้างแจ้งเตือนตัวอย่าง ให้ buyer seed ด้วย ถือเป็นข้อมูลสาธิต ไม่ใช่ขยะ

---

5. Trigger 4: trg_refresh_product_quantity

5.1 หน้าที่และเหตุผล

product.quantity ต้องเท่ากับผลรวมสต็อกทุก variant ของสินค้านั้นเสมอ
ถ้าให้โค้ดเว็บคำนวณเองทุกครั้งที่เพิ่ม/แก้/ลบ variant จะมีจังหวะหลุด (เช่น สร้าง variant แล้วลืมบวกยอด หรือลบแล้วลืมหัก) Trigger นี้ทำให้ยอดรวม ถูกต้องโดยไม่ต้องไว้ใจผู้เรียก

นิยามใน repo: supabase/migrations/0005_product_variants.sql:73-98

```
create or replace function public.refresh_product_quantity()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  if TG_OP = 'DELETE' then
    update public.product p
    set quantity = coalesce(
      (select sum(quantity) from public.product_variant v
       where v.product_id = OLD.product_id), 0)
    where p.product_id = OLD.product_id;
    return OLD;
  else
    update public.product p
    set quantity = coalesce(
      (select sum(quantity) from public.product_variant v
```

```
            where v.product_id = NEW.product_id), 0)
        where p.product_id = NEW.product_id;
        return NEW;
    end if;
end;
$$;

drop trigger if exists trg_refresh_product_quantity on public.product_variant;
create trigger trg_refresh_product_quantity
after insert or update of quantity, product_id or delete on public.product_variant
for each row execute function public.refresh_product_quantity();
```

5.2 อธิบายโค้ด

1. แยกสาขา DELETE (ใช้ OLD.product_id) กับ INSERT/UPDATE (ใช้ NEW.product_id) เพราะใน trigger DELETE ตัวแปร NEW เป็น null
2. ใช้ coalesce(..., 0) ถ้าลบ variant จนหมด สินค้าจะเหลือ 0 ไม่ใช่ null
3. ดักเฉพาะคอลัมน์ quantity, product_id ตอน UPDATE การแก้แค่ size/color ไม่ทำให้ trigger ทำงานเปลือง (แต่เขียนทับทั้งแถวจากหน้าเว็บก็ยังติด trigger อยู่ดีเพราะ quantity รวมอยู่ใน SET เสมอ — ปลอดภัยไว้ก่อน)
4. migration เดียวกันมี backfill สร้าง variant ให้สินค้าเดิม + sync ยอดครั้งเดียว ตอนรัน (update product set quantity = coalesce(sum...)) จึงไม่มีสินค้ายอดค้าง

5.3 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่คาด (จากโค้ด)	สถานะการยืนยัน
4.1 เพิ่ม variant ใหม่ (สต็อก 5)	เพิ่มผ่านหน้าแก้ไขสินค้า แล้วดู product.quantity	เพิ่มขึ้น 5	ยืนยันแล้ว (สินค้าเพิ่มจาก 10 เป็น 12 ชิ้น ยอดรวมขยับตาม)
4.2 แก้สต็อก variant	แก้จำนวนแล้วดูยอดรวม	ยอดรวม = ผลรวมใหม่	ตามโค้ด
4.3 ลบ variant ตัวสุดท้าย	ลบแล้วดูยอดรวม	เหลือ 0 (ไม่เป็น null)	ตามโค้ด
4.4 แก้แค่ size/color	แก้ซื่อสีแล้วดูยอดรวม	ยอดเท่าเดิม	ตามโค้ด

[FIG 5: เทียบยอด product.quantity กับผลรวม variants ใน Table Editor]

5.4 หน้าจอที่เกี่ยวข้อง

หน้าเพิ่ม/แก้ไขสินค้าฝั่งผู้ขาย (SellerProductEditPage.vue ส่วนจัดการ variants) + ตารางจัดการสินค้าที่โชว์ยอดรวม

5.5 ข้อสังเกต

- product.quantity ยังเป็นคอลัมน์จริง (ไม่ใช่ generated column) เพราะ Postgres ไม่รองรับ subquery ใน generated column จึงต้องใช้ trigger รักษายอดแทน
- คอลัมน์ product.size/color เดิมยังอยู่แต่ไม่มีโค้ดเขียนแล้ว (legacy) เก็บไว้เพื่อ ข้อมูลเก่า

6. Trigger 5: trg_messages_notify

6.1 หน้าที่และเหตุผล

เมื่อมีข้อความแชทใหม่ต้องแจ้ง “อีกฝ่าย” (ไม่ใช่คนส่ง) เพราะผู้รับไม่มีทางรู้ว่ามีคนทัก จนกว่าจะเปิดห้องแชทดูเอง ก่อนหน้านี้ไม่มีอะไรสร้าง notification ตอนแชทเลย การแจ้งเตือนแบบ realtime ฝั่งเว็บทำงานเฉพาะตอนเปิดเว็บค้างไว้เท่านั้น แถวนี้ใน DB ทำให้เปิดมาเห็นย้อนหลังได้

นิยามใน repo: supabase/migrations/0009_chat_message_notify.sql (สาขา messages ใน notify_user() +

คอลัมน์ notifications.link)

```
drop trigger if exists trg_messages_notify on public.messages;
create trigger trg_messages_notify
after insert on public.messages
for each row execute function public.notify_user();
-- ใน notify_user(): สาขา tg_table_name = 'messages'
-- หา buyer/seller ของห้องจาก new.conversation_id
-- ถ้าผู้ส่งคือ buyer แจ้ง seller (และกลับกัน) ชื่อผู้ส่ง fallback ผู้ซื้อ/ผู้ขาย
-- insert (user_id=อีกฝ่าย, type='chat', title='ข้อความใหม่จาก {ชื่อ}',
--         content=left(new.content,120), link=conversation_id)
```

6.2 อธิบายโค้ด

1. หาคู่สนทนาจากห้อง ถ้าห้องไม่มี (ไม่ควรเกิดเพราะ FK) ข้ามเงียบๆ ไม่พังการส่งข้อความ
2. ระบุผู้รับ = อีกฝ่ายที่ไม่ใช่ผู้ส่ง แล้วดึงชื่อผู้ส่งจากตารางตามบทบาท (ชื่อบังคับตอนสมัครอยู่แล้ว กรณีว่างใช้คำว่า "ผู้ซื้อ"/"ผู้ขาย")
3. content ตัด 120 ตัวอักษร (snapshot ตั้งแต่วินาทีที่ส่ง) ส่วน link เก็บ id ห้องไว้ ให้ frontend กดแล้วเด้งเข้าห้องถูกฝั่งตามบทบาท
4. กันผู้ส่งแปลกหน้า (v_to เป็น null) ข้ามไป ไม่สร้างแจ้งเตือนมั่ว

6.3 ผลการทดสอบ

กรณี	วิธีตรวจ	ผลที่คาด (จากโค้ด)	สถานะการยืนยัน
5.1 ข้อความใหม่ถึงอีกฝ่าย	ทักจากอีกบัญชี แล้วดู notifications ของผู้รับ	มีแถว type='chat' title ถูก + link ตรงห้อง	รอผลรัน
5.2 ผู้ส่งไม่ได้รับแจ้งเตือนตัวเอง	ดู notifications ของผู้ส่ง	ไม่มีแถวใหม่	ตามโค้ด
5.3 ผู้รับเปิดห้องนั้นค้างไว้	ทักขณะที่อีกฝ่ายเปิดห้องนั้นอยู่	แถวแจ้งเตือนยังเกิด (frontend ไม่เด้ง toast ซ้ำ)	ตามโค้ด มีข้อสังเกต

[FIG 6: กระดิ่งขึ้น "ข้อความใหม่จาก ..." เมื่ออีกฝ่ายทักแชท]

6.4 หน้าจอที่เกี่ยวข้อง

กระดิ่งทั่วแอป (SiteNavbar.vue/SellerLayout.vue โหลด + subscribe realtime) + หน้าแจ้งเตือน (NotificationsPage.vue กดแถว chat แล้วเข้าห้องตามบทบาท) + toast ใน session (stores/chat.js เตือนเฉพาะตอนเปิดเว็บค้างและไม่ได้ดูห้องนั้นอยู่)

6.5 ข้อสังเกต

- เตือนทุกข้อความแม้ผู้รับกำลังดูห้องนั้นอยู่ แถวใน DB เกิดเสมอ ฝั่งเว็บแก้ด้วยการ ไม่เด้ง toast ซ้ำในห้องที่เปิดอยู่แล้ว ถ้าอนาคตแชทเยอะควรเตือนเฉพาะข้อความแรก ของห้องที่ยังไม่อ่าน (ดูข้อเสนอแนะข้อ 1 ประกอบ)

7. สรุปผลการทดสอบ

Trigger	จำนวนกรณี	ตรงตามโค้ด	หมายเหตุ
1. trg_order_item_notify	3	3	1.1 ยืนยันจากโฟลว์จริงแล้ว
2. trg_orders_status_notify	3	3	2.1 รอผลรัน
3. trg_orders_insert_notify	2	2	3.1 รอผลรัน
4. trg_refresh_product_quantity	4	4	4.1 ยืนยันจากข้อมูลจริงแล้ว

Trigger	จำนวนกรณี	ตรงตามโค้ด	หมายเหตุ
5. trg_messages_notify	3	3	5.1 รอผลรัน, 5.3 มีข้อสังเกต
รวม	15	15	รอผลรัน 5 กรณี (1.2, 2.1, 3.1, 3.2, 5.1)

กรณีหลักที่ Trigger ตั้งใจทำ (แจ้งเตือนถูกคน/ถูกจังหวะ, ยอดสต็อกรวมตรง) ได้รับการยืนยันจากโฟลว์ใช้งานจริงแล้ว ส่วนกรณีที่เหลือเป็นไปตามโค้ดที่ทบทวนแล้ว

8. ข้อเสนอแนะ

1. Trigger แชทเตือนแม้ผู้รับกำลังดูห้องอยู่ ดูรายละเอียดในหัวข้อ 6.5 (แถวแจ้งเตือนเกิดทุกข้อความ ถ้าอนาคตแชทเยอะควรเตือนเฉพาะข้อความแรก ของห้องที่ยังไม่อ่าน)
2. notifications ไม่มี INSERT policy โดยตั้งใจ หน้าเว็บเขียนแจ้งเตือนตรงไม่ได้ ต้องผ่าน Trigger/RPC เท่านั้น ถ้าวันหนึ่งต้องให้ระบบอื่นเขียนแจ้งเตือน ให้เพิ่ม policy แบบจำกัดแทนการเปิดกว้าง
3. ชื่อผู้ส่งแชท fallback ถ้า buyer/seller ไม่มีชื่อ จะใช้คำว่า "ผู้ซื้อ"/"ผู้ขาย" ปัจจุบันชื่อบังคับตอนสมัครอยู่แล้ว กรณีนี้เกิดยาก
4. ข้อความแชทในแจ้งเตือนเป็น snapshot 120 ตัวอักษร ถูก freeze ตั้งแต่ตอนส่ง (ข้อความในแอปแก้ไม่ได้อยู่แล้ว จึงไม่มีปัญหา)
5. ห้ามลบสินค้าที่เคยถูกสั่ง (order_item.product_id เป็น RESTRICT) เป็นการออกแบบ เพื่อกันประวัติพัง ถ้าอยากซ่อนสินค้าแทนลบ ต้องเพิ่มคอลัมน์สถานะขายแทน

9. วิธีการทดสอบและที่มาของภาพ

- นิยาม Trigger และฟังก์ชันทั้งหมดดึงจาก migration ใน repo (supabase/migrations/) ซึ่งเป็น source of truth เดียวกับที่รันบนฐานข้อมูลจริง
- กรณีที่ต้องเขียนข้อมูลทดสอบ จะทดสอบบนโปรเจกต์ทดสอบแยกเท่านั้น ไม่รันบนฐานข้อมูลจริงที่มีข้อมูลขาย (สคริปต์เทสแยกไม่ทำตามคำขอ)
- ภาพหน้าจอแอปถ่ายจากโค้ดหน้าเว็บจริงใน repo (Vite dev / Vercel) ข้อมูลในภาพเป็น ข้อมูลทดสอบ ไม่ใช่ข้อมูลผู้ใช้จริง
- คำสั่งตรวจแบบอ่านอย่างเดียวที่ใช้ยืนยัน:
- รายการ trigger: select tgname, tgrelid::regclass from pg_trigger where not tgisinternal order by 2, 1;
- ยอดรวมตรงไหม: select p.product_id, p.quantity, coalesce(sum(v.quantity), 0) as sum_v from product p left join product_variant v on v.product_id = p.product_id group by p.product_id, p.quantity having p.quantity is distinct from coalesce(sum(v.quantity), 0); (ได้ 0 แถว = ตรงทุกชิ้น)

ภาคผนวก: รายชื่อไฟล์ภาพ

ภาพที่	ไฟล์	เนื้อหา
1	fig01_trigger_list.png	รายการ Trigger ทั้งหมด ดึงจาก pg_trigger ผ่าน SQL Editor
2	fig02_order_placed.png	กระดิ่งผู้ซื้อขึ้น "บันทึกคำสั่งซื้อแล้ว" หลังกดสั่ง
3	fig03_status_thai.png	กระดิ่งผู้ซื้อขึ้นป้ายไทยหลังผู้ขายอัปเดตสถานะ
4	fig04_seller_order.png	กระดิ่งผู้ขายขึ้น "มีออเดอร์ใหม่"
5	fig05_qty_check.png	เทียบยอด product.quantity กับผลรวม variants
6	fig06_chat_notify.png	กระดิ่งขึ้น "ข้อความใหม่จาก ..." เมื่ออีกฝ่ายทักแชท